



Artificial Intelligence Software, HW 1

Reproducible environment setup:
Conda, UV, Docker



Center for
Cognitive
Modeling



Task 1

Short description: Describe your experience with vibecoding (AI coding assistants).

If you have no prior experience, describe your expectations or the reasons why you haven't tried it yet (instead of answering the Requirements section below).

Artifacts: Report in pdf format.

Requirements

1. What tasks did you try to solve using vibecoding? Which tools did you use?
2. What did you like and dislike about the tools?
3. What challenges or hallucinations did you encounter?
4. How did you verify the correctness of the generated code?
5. Which tools would you recommend to a friend and why?
6. What would you like to learn about vibecoding in the next lecture?



Task 2

Short description: Build a Docker image with ROS 2 Humble based on ubuntu:jammy with GPU, GUI, and host file access support.

Artifacts: Dockerfile, docker-compose.yml, pdf file with brief explanation of your solution.
Requirements

- Base image: ubuntu:jammy
- ROS 2 Humble via official ROS repositories
- UID/GID mapping for host file access
- Start container using docker compose
- Pass ROS_DOMAIN_ID as environment variable
- Mounted volume for verification
- network_mode: host
- NVIDIA Container Toolkit support (nvidia-smi)
- GUI support (activate rviz2)
- Container stays alive after docker compose up (can be solved via *command*, *tty* or *entrypoint*)
- Interactive access via docker exec -it



Task 3

Short description: set up and compare a Data Science environment using 4 different tools: **Conda**, **Poetry**, **uv**, and **Pixi**.

Artifacts: Repository with configuration folders for each tool, benchmark.py script, and report in pdf format with comparison. Compare **Cold Install Time**, **Disk Usage**, and developer experience.

Requirements

1. **Benchmark Script:** Create benchmark.py that uses numpy, pandas, matplotlib, plotly to generate data and save a plot.
2. **Conda:** Create env from environment.yaml, export full lock-file.
3. **Poetry:** Init project, resolve dependencies, run script via poetry run.
4. **uv:** Create venv, install deps. Focus on resolution speed.
5. **Pixi:** Init project. Install Python deps + one system tool (e.g. ffmpeg or graphviz).

Deserve "A" grade!

– Oleg Bulichev

✉ bulichev.ov@mipt.ru

📍 @Lupasic

🏠 Цифра, 521